Assignment 2 Report

Parallel Programming Using CUDA and OpenMP

Local Histogram Equalization on 3D Point Clouds

Himanshi Bhandari
Roll No: 2023CS10888
Course: COL380

March 28, 2026

Contents

1 Introduction

This assignment studies local histogram equalization on 3D point clouds using three different neighborhood definitions: exact KNN, approximate KNN, and K-means clustering. The goal is to produce enhanced intensities for every point while comparing the runtime behavior of the three approaches under different values of n, k, and T.

The implementation uses CUDA for GPU acceleration and OpenMP for CPU-side preprocessing. The assignment also requires a comparison of runtime across the three histogram-building strategies and a discussion of the trade-off between speed and accuracy. The K-means method must stop either when assignments stop changing or when the maximum iteration count T is reached.

2 Problem Statement and Equalization Formula

A 3D point cloud is a set

$$P = \{p_1, p_2, \ldots, p_n\},$$

where each point has coordinates (x_i, y_i, z_i) and intensity $I_i \in \{0, \ldots, 255\}$.

For each point p_i, a neighborhood $N(i)$ is selected by one of the three algorithms. The local histogram is

$$H_i[v] = |\{j \in N(i) : I_j = v\}|, \qquad v \in \{0, \ldots, 255\}.$$

The local CDF is

$$C_i[v] = \sum_{u=0}^{v} H_i[u].$$

Let

$$C_{i,\min} = \min\{C_i[v] \mid C_i[v] > 0\}.$$

The equalized intensity is

$$I_i' = \left\lfloor \frac{C_i[I_i] - C_{i,\min}}{m - C_{i,\min}} \cdot 255 \right\rfloor,$$

where m is the neighborhood size. For exact KNN and approximate KNN, $m = k$; for K-means, m is the cluster size. If $m = C_{i,\min}$, the original intensity is retained.

In the KNN-based methods, the implementation computes the result directly from the selected neighbor intensities. The neighborhood always includes the query point itself, which is why the code treats the effective size as $k + 1$ when equalizing. In K-means, the equalization uses the full histogram of the assigned cluster.

3 Implementation Overview

The implementation is organized into three CUDA-driven pipelines:

1. **Exact KNN**: every point searches all other points and keeps the k nearest neighbors using a bounded max-heap.
2. **Approximate KNN**: the host builds a spatial grid using OpenMP, and each GPU thread searches nearby grid cells using expanding shells.
3. **K-means**: centroids are initialized from the first k points, assignments are updated on the GPU, and the algorithm stops when assignments stop changing or when T iterations are reached.

The output of each method is written to a separate file in the original input order: `knn.txt`, `approx_knn.txt`, and `kmeans.txt`.

3.1 Implementation Optimizations

Several low-level optimizations were used in addition to the main algorithmic design.

Compiler optimization flags. The code was compiled with aggressive optimization enabled, including the `-O3` flag. This helps the compiler perform loop unrolling, function inlining, strength reduction, and other instruction-level optimizations that improve execution speed in the hot paths of the program.

Loop unrolling. In the performance-critical inner loops, loop unrolling was used either explicitly through compiler directives or implicitly through optimization at higher compilation levels. This reduces loop-control overhead and exposes more instruction-level parallelism, especially in repeated distance-computation and histogram-processing sections.

OpenMP parallelization. The CPU-side preprocessing stages were parallelized using OpenMP pragmas. In particular, the bounding-box computation and related initialization steps were implemented with `#pragma omp parallel for` and reduction-based clauses wherever applicable. This allows multiple CPU threads to work on the input point cloud simultaneously before the GPU kernels are launched.

Read-only cache optimization. For data that are only read inside the kernels, such as point coordinates and intensities, the implementation uses the GPU read-only cache path through __ldg where possible. This improves memory behavior because repeated reads of the same input values can be served from the read-only cache instead of global memory.

Shared-memory reuse. The exact KNN and K-means implementations both reuse data through shared memory, reducing repeated global-memory traffic. In exact KNN, tiles of points are loaded once and reused by all threads in a block; in K-means, the current centroids are cached in shared memory for fast access by all threads in the block.

Why these optimizations matter. These optimizations do not change the asymptotic complexity of the algorithms, but they significantly reduce constant factors. In practice, they improve memory locality, lower global-memory traffic, and reduce the number of expensive operations inside the kernels. This is especially important for approximate KNN, where the speedup comes from combining better algorithmic pruning with careful memory-level optimization.

4 Algorithm 1: Exact KNN

4.1 Algorithm Description

For each query point p_i, exact KNN computes the squared Euclidean distance to every other point:

$$d(p_i, p_j) = (x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2.$$

The k smallest distances are retained in a bounded max-heap. The root of the heap stores the current worst among the selected neighbors, so a candidate can be discarded quickly if it is farther than the current heap maximum.

The kernel processes the point cloud in tiles. A tile of points is copied into shared memory, and then every active thread compares its query point against that tile. This reduces repeated global-memory access while still examining all points.

After the final neighbor set is formed, the equalization step is applied using the intensities of the selected neighbors.

4.2 Parallelization Strategy and Design Decisions

The exact KNN implementation uses the following design decisions:

- **One CUDA thread per query point.** Each thread computes the neighborhood for one point independently, so the problem is embarrassingly parallel across points.
- **Shared-memory tiling.** Points are processed in blocks of size (tile size)=1024. Each block loads one tile into shared memory and reuses it across all threads in the block.
- **Bounded max-heap.** The heap stores only the best k candidates. Heap insertions and updates cost $O(\log k)$.
- **Pruning using the current heap maximum.** Once the heap is full, any candidate with distance larger than the current maximum can be skipped without a heap operation.
- **Read-only access path.** Coordinate and intensity reads use __ldg where possible, which helps cache utilization.
- **Deterministic tie-breaking.** If two candidates have the same distance, the code resolves the tie lexicographically by (x, y, z). This makes the output stable.

4.3 Complexity

For one query point, exact KNN scans all n points and may insert up to k of them into the heap. The worst-case cost per point is

$$O(n \log k).$$

Therefore the total cost is

$$O(n^2 \log k).$$

The tiling strategy improves memory behavior and constant factors, but it does not change the asymptotic complexity. The equalization stage costs $O(k)$ per point, which is lower than the search cost, so the total remains

$$O(n^2 \log k).$$

4.4 Why the Graphs Behave This Way

The exact KNN runtime increases sharply with n because every point compares against the entire dataset. It also increases with k because heap maintenance becomes more expensive and the equalization stage must process a larger neighborhood. It does not depend on T.

5 Algorithm 2: Approximate KNN

5.1 Algorithm Description

Approximate KNN speeds up the search by restricting it to spatially nearby points. The implementation has two stages.

5.1.1 Host-side grid construction

First, the host computes the bounding box of the point cloud using OpenMP reductions:

$$\min(x), \max(x), \min(y), \max(y), \min(z), \max(z),$$

which correspond to the variables `minX, maxX, minY, maxY, minZ, maxZ` in the implementation.

Then, a cubic grid resolution is selected. The cell size (`cellSize`) is estimated based on the point-cloud volume and the desired average occupancy. The goal is to ensure that each grid cell contains only a small number of nearby points, thereby reducing the search space from global to local.

Each point is mapped to a 3D grid cell using the array `id_of_cell`. The grid is then constructed using a counting-sort style approach with the following arrays:

- `ord`: reordered point indices grouped by cell,
- `H_start`: starting offset of each cell in `ord`,
- `H_end`: ending offset of each cell in `ord`.

This is essentially a counting-sort style layout. Points in the same cell are stored contiguously, which makes the GPU search simpler and faster.

5.1.2 GPU-side neighborhood search

For each query point, the kernel:

1. computes the cell containing the query point,
2. searches that cell first,
3. expands to neighboring shells around the cell,
4. keeps only the best candidates in a heap,
5. stops early if the current k-th best candidate is already guaranteed to be closer than any point in the remaining unexplored region.

The search starts with a small shell radius. In the code, the initial shell radius is 2, and it can grow adaptively up to 8 if the kernel has not yet collected enough candidates. This is a key optimization: it avoids searching a very large region unless the local neighborhood is too sparse.

The equalization stage is then applied exactly as in exact KNN.

5.2 Parallelization Strategy and Design Decisions

Approximate KNN is the most important optimization in the assignment, and the code uses several design choices to make it fast:

- **OpenMP preprocessing on the host.** Bounding-box computation and cell assignment are parallelized on the CPU side.
- **Spatial hashing through grid cells.** Instead of comparing against all points, the method only inspects points in nearby cells.
- **Contiguous cell ranges.** The arrays `ord`, `H_start`, and `H_end` allow the kernel to iterate over points within a cell using simple linear scans.

- **Shell-based expansion.** The algorithm expands outward in 3D shells rather than searching the whole space. This keeps the search local.
- **Adaptive shell limit.** If the current neighborhood is too small, the shell limit is increased gradually, up to a fixed cap.
- **Candidate pruning with a bounded heap.** Only points better than the current heap maximum are inserted.
- **Lower-bound early termination.** Once the k-th best distance is smaller than a geometric lower bound for all remaining unexplored cells, the kernel stops.
- **Same equalization logic as exact KNN.** This keeps the output pipeline consistent across the two KNN-based methods.

5.3 Complexity

The approximate method has two parts.

Preprocessing on the host. The bounding-box computation is $O(n)$ with OpenMP. The cell assignment is also $O(n)$. The prefix-sum style construction over the grid costs

$$O(n + C),$$

where $C = \text{gridDimX} \cdot \text{gridDimY} \cdot \text{gridDimZ}$ is the number of cells. In the code, the grid is further constrained so that C stays below a fixed cap of about 8 million.

Search on the GPU. For one query point, the kernel scans only a local neighborhood of cells. The shell traversal itself is small because the shell radius is capped, but the number of candidate points depends on the local density of the point cloud.

So the query cost is data-dependent:

$$O(c \log k),$$

where c is the number of candidate points actually examined in the visited shells.

In the worst case, if many points lie in the visited cells or if the grid is poorly balanced, the method can degenerate toward the exact-search behavior. But in the intended setting, the practical runtime is much closer to local-neighborhood cost than to global-search cost.

Including the final equalization step, the per-point runtime remains dominated by candidate search and heap maintenance.

5.4 Why the Graphs Behave This Way

Approximate KNN grows much more slowly than exact KNN because it does not scan all n points for every query. Increasing k makes the heap slightly more expensive and may force the shell search to continue a bit longer, but the increase is still far smaller than in exact KNN. The runtime does not depend on T.

6 Algorithm 3: K-means Clustering

6.1 Algorithm Description

K-means begins by choosing the first k points as initial centroids. Then it repeats the following steps:

1. **Assignment:** each point is assigned to the nearest centroid.
2. **Accumulation:** for each cluster, the sums of coordinates and the cluster sizes are computed.
3. **Update:** each centroid is replaced by the average of its assigned points.

The algorithm stops as soon as no point changes its cluster, or when T iterations are completed.

After clustering, the code builds a histogram for every cluster and then equalizes each point using the histogram of its assigned cluster.

6.2 Parallelization Strategy and Design Decisions

The K-means implementation uses the following design choices:

- **One thread per point for assignment.** Every point independently finds the nearest centroid.
- **Shared-memory centroid cache.** Each block loads the current centroids into shared memory so that all threads in the block can reuse them.
- **Atomic convergence flag.** If any point changes its cluster, a device-side flag is set.
- **Atomic accumulation of cluster sums.** Sums and counts are updated in parallel using atomicAdd.
- **Early termination.** The host checks the convergence flag and stops before reaching T if the assignments are stable.
- **Histogram build after clustering.** Once the final clusters are known, the cluster histograms are computed and used for equalization.
- **Pinned host memory for output transfer.** The final intensities are copied back using pinned memory to reduce transfer overhead.

6.3 Complexity

For each iteration, the assignment kernel performs one distance check against each of the k centroids for every point:

$$O(nk).$$

The accumulation step is

$$O(n),$$

and centroid recomputation is

$$O(k).$$

If the algorithm runs for t actual iterations before convergence, the clustering stage costs

$$O(tnk).$$

Since $t \leq T$, the upper bound is

$$O(Tnk).$$

The histogram-building stage after clustering is $O(n)$, and the final equalization stage is also linear up to a constant factor of 256:

$$O(n).$$

So the overall asymptotic runtime remains

$$O(tnk + n),$$

which is usually written as

$$O(Tnk)$$

for the full upper bound.

6.4 Why the Graphs Behave This Way

K-means is almost flat with respect to T in my experiments because the algorithm converges before the iteration cap is reached. Runtime increases slowly with n because each iteration is linear in the number of points. Increasing k adds more centroid comparisons, but the impact is much smaller than in exact KNN.

7 Test Setup

All experiments were conducted on the IIT Delhi HPC cluster equipped with NVIDIA GPUs (Kepler `sm_35` architecture). The code was compiled using `nvcc` from the NVIDIA HPC SDK 20.7 with CUDA 11.0.

Compiler Optimization. In addition to the parallelization strategies, i also tried compiling the code using the `-O3` optimization flag to enable aggressive compiler-level optimizations such as loop unrolling, inlining, and instruction-level optimizations for improved performance.

The benchmark dataset used for the main speedup measurement was a randomly generated point cloud with

$$N = 100000, \qquad K = 128, \qquad T = 50.$$

The runtimes were measured using the shell `time` command, so they are wall-clock execution times.

8 Performance Analysis

8.1 Runtime Variation with T

Table 1: Runtime variation with T.

T	KNN (ms)	Approx KNN (ms)	K-means (ms)
1	3405.39	667.91	151.52
5	3213.59	664.74	151.90
10	3213.75	666.80	155.56
25	3242.45	663.38	162.60
50	3211.81	666.42	176.67

KNN and approximate KNN do not depend on T, so their runtimes stay essentially flat. K-means remains almost constant because it converges early; increasing T only matters when the clustering actually needs more iterations.

8.2 Runtime Variation with n

Table 2: Runtime variation with n.

n	KNN (ms)	Approx KNN (ms)	K-means (ms)
10000	305.00	137.97	103.35
25000	680.37	209.02	115.33
50000	1499.04	355.22	135.56
75000	2298.30	509.80	155.29
100000	3194.87	663.70	186.34

All three methods slow down as n grows, but exact KNN grows the fastest because it compares every point against almost every other point. Approximate KNN is much better because it only inspects nearby cells. K-means grows more gently because each iteration is only $O(nk)$ and the number of iterations is small in practice.

8.3 Runtime Variation with k

Table 3: Runtime variation with k.

k	KNN (ms)	Approx KNN (ms)	K-means (ms)
1	467.17	168.74	162.19
2	451.11	166.40	153.08
4	462.53	172.84	156.33
8	491.45	185.93	154.38
16	595.38	206.83	158.18
32	856.40	258.81	166.84
64	1549.93	390.31	165.70
128	3259.08	668.09	176.26

The exact KNN runtime increases sharply with k because the heap is larger and more heap operations are needed. Approximate KNN also increases with k, but more gradually because the candidate set is already reduced by the grid. K-means changes only slightly with k because it mainly compares against k centroids, and the rest of the pipeline is dominated by the linear scan over points.

8.4 Experimental Plots

The following plots summarize the observed behavior of the three methods.

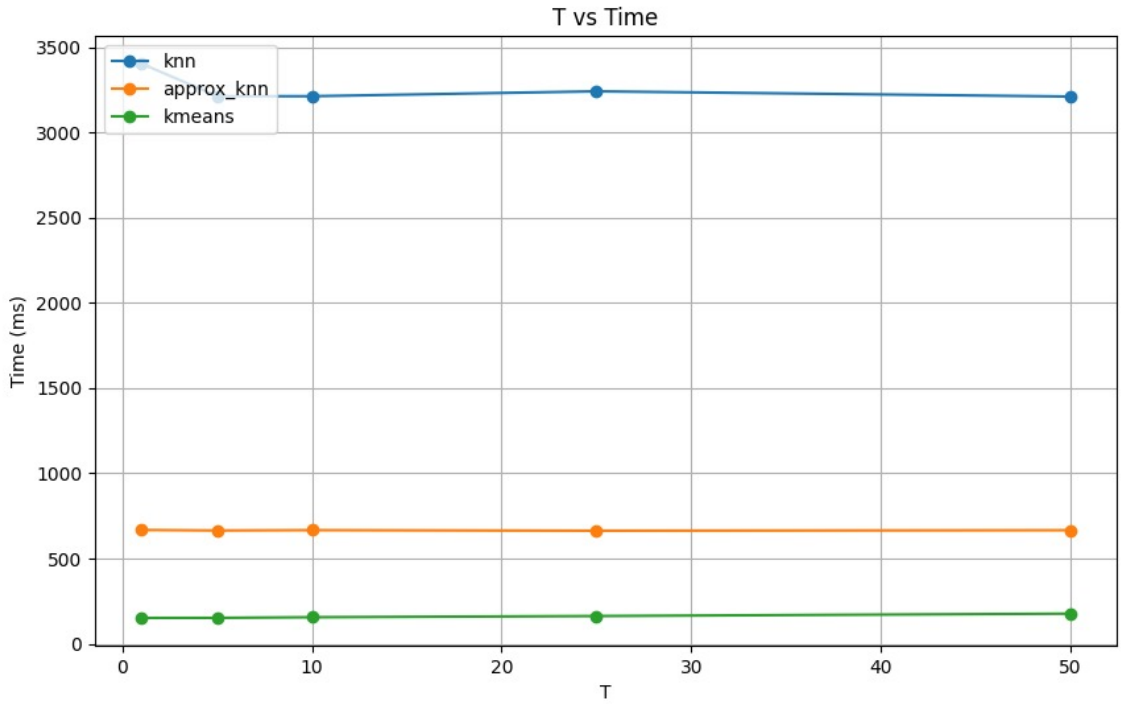

Figure 1: Runtime variation with T.

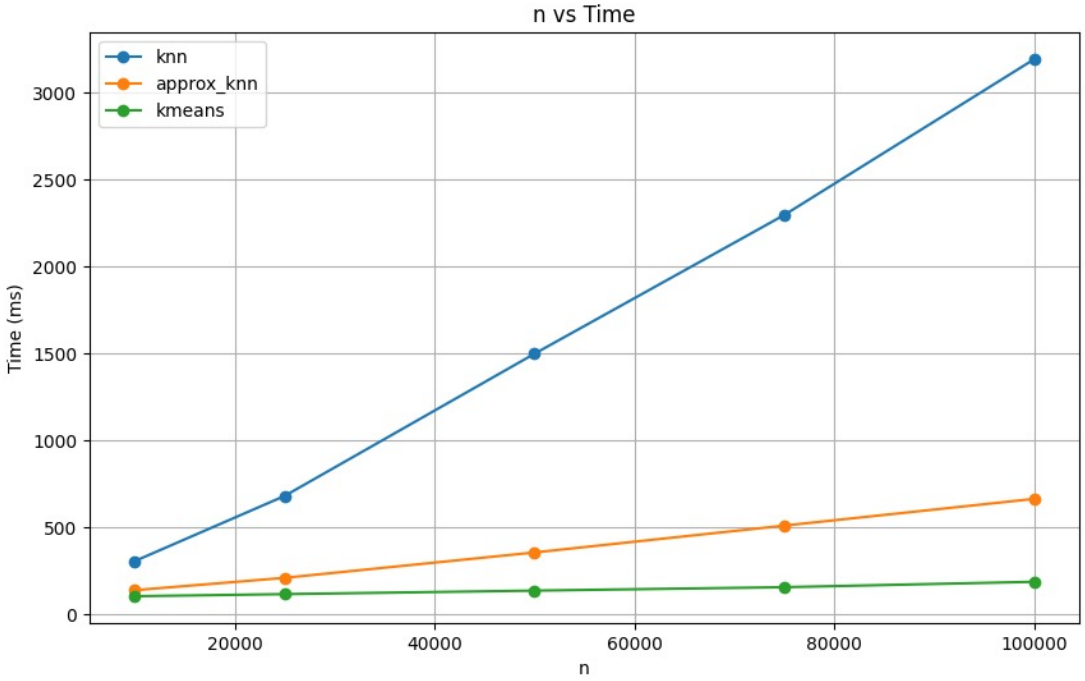

Figure 2: Runtime variation with n.

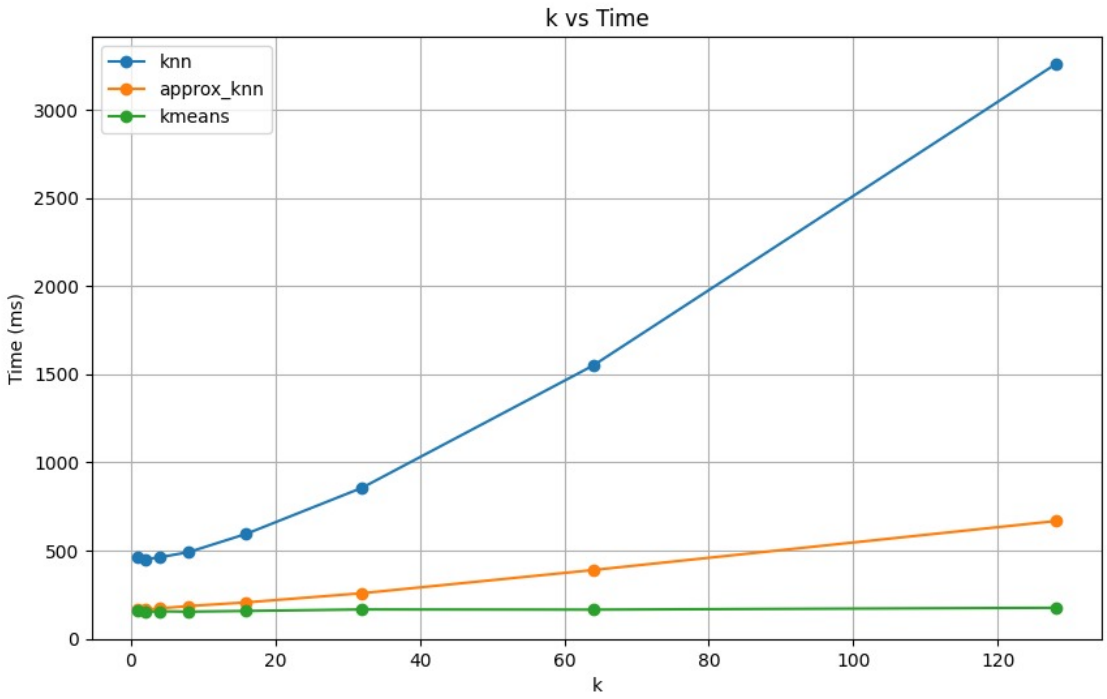

Figure 3: Runtime variation with k.

8.5 Observed Speedups on the Benchmark Dataset

The following runtimes were measured on the dataset with $N = 100000$, $K = 128$, and $T = 50$.

Table 4: Sequential versus parallel runtimes and speedups.

Algorithm	Sequential Time (s)	CUDA/OpenMP Time (s)	Speedup
KNN	20.876	3.417	6.11×
Approx KNN	20.876	0.691	30.21×
K-means	10.715	0.215	49.84×

The speedup is computed as

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}.$$

For the KNN-family methods, the sequential KNN runtime is used as the baseline because approximate KNN solves the same neighborhood-selection task but with a reduced search space. For K-means, the sequential K-means runtime is used as the baseline.

8.6 Explanation of the Speedups

Exact KNN achieves about 6.11× speedup because the GPU parallelizes the per-point search, but each thread still performs a large amount of work. Approximate KNN achieves about 30.21× speedup because the grid dramatically reduces the number of candidate points. K-means achieves the largest speedup, about 49.84×, because each point only compares against k centroids and the algorithm converges early.

Mostly the relative ordering observed by me of runtimes is

$$\text{K-means fastest} \;<\; \text{Approx KNN} \;<\; \text{Exact KNN slowest}.$$

8.7 Accuracy Analysis (MAE)

To evaluate the quality of the approximate KNN method, we compute the Mean Absolute Error (MAE) between the output intensities of approximate KNN and exact KNN.

The MAE is defined as:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |I_i^{\text{approx}} - I_i^{\text{exact}}|.$$

For a particular input with largest benchmark configuration ($N = 100000$, $K = 128$, $T = 50$), the observed MAE was:

$$\text{MAE} = 0.001820.$$

Across multiple input configurations with varying values of n, k, and T, the MAE consistently remained very low:

$$\text{MAE} < 0.01.$$

Interpretation. The extremely small MAE indicates that the approximate KNN method closely matches the exact KNN results while achieving significantly better performance. This confirms that the spatial grid and shell-based search effectively preserve local neighborhood structure with minimal loss in accuracy.

Thus, approximate KNN provides an excellent trade-off between accuracy and runtime efficiency.

9 Discussion of the Graphs

9.1 Why the exact KNN curve grows fastest

Exact KNN performs a full global scan for every query point. As n and k increase, the number of distance calculations and heap updates increases rapidly, so the runtime rises sharply.

9.2 Why approximate KNN is much flatter

Approximate KNN does not inspect all points. It only searches nearby grid cells and expands outward when needed. This reduces the number of comparisons substantially, which is why the curve is much flatter.

9.3 Why K-means is nearly constant in T

K-means stops as soon as assignments stabilize. In my experiments, convergence happens before the maximum iteration count, so increasing T does not change runtime much.

9.4 Why increasing k hurts exact KNN the most

Exact KNN must maintain a larger heap, so heap maintenance becomes expensive. Approximate KNN is less sensitive because the search space is already limited. K-means is the least sensitive because it only compares each point against the current centroids.

10 Conclusion

The three methods provide a clear trade-off between accuracy and performance. Exact KNN gives the most faithful neighborhood but is the slowest. Approximate KNN improves runtime substantially by limiting the search to nearby spatial cells. K-means is the fastest method because it replaces neighborhood search with centroid assignment and often converges in few iterations.

For the given input used for testing, the benchmark results confirm the effectiveness of the CUDA/OpenMP implementation:

- Exact KNN: 6.11× speedup,
- Approximate KNN: 30.21× speedup,
- K-means: 49.84× speedup.

These results show that the design decisions in the implementation substantially reduce runtime while preserving the intended functionality of the assignment.